Deep Learning

Homework 4

Iman Jamshidi

Fall2025

Step 1: Data

1) **How many data samples are included in the dataset?**
   The dataset has 600 graphs total. Each graph represents one enzyme structure. So basically I'm working with 600 different molecular structures here.
2) **Which problem will this dataset try to address?**
   This is a graph classification problem. The goal is to classify enzymes into 6 different types based on their molecular structure. So given a graph representation of an enzyme, the model needs to predict which of the 6 enzyme classes it belongs to. Pretty straightforward classification task but with graph-structured data instead of regular images or text.
3) **What are nodes, edges and features in your dataset?**
   The nodes represent parts of the enzyme molecules - I think they're like atoms or functional groups in the structure. Each graph has different number of nodes, ranging from small ones to larger molecular structures.
   The edges show connections between these nodes - basically which parts of the molecule are connected to each other. The dataset gives me an adjacency matrix for this. I converted it to undirected edges because molecular bonds work both ways.
   For features, each node has 18 continuous attributes plus 1 categorical node label. So total 19 features per node. I combined the node attributes and node labels together as the input features. Not sure exactly what these 18 attributes represent chemically, but they're numerical values that describe properties of each node.

4) **Does the dataset have any missing information?**
   No missing data that I found. All 600 graphs have complete information - every node has its 19 features, all edges are specified, and every graph has a label. The dataset is pretty clean actually, didn't need to deal with any NaN values or missing entries.

5) **What is the label of this dataset?**
   Labels are integers from 1 to 6, representing 6 different enzyme classes. I converted them to 0-5 range for the model (zero-indexed). Each of the 600 graphs belongs to exactly one of these 6 classes. The classes seem roughly balanced - not perfectly equal but no extreme imbalance either.
6) **How many percent of data will you use for training, validation and testing?**
   I split the data 80/10/10. So 480 graphs for training (80%), 60 for validation (10%), and 60 for test (10%).
   Used random split with seed=42 for reproducibility. The validation set helps me tune hyperparameters and do early stopping during training. Test set is held out completely until final evaluation.

7) **What kind of data pre-processing will you use for your training dataset?**

Few preprocessing steps I applied:

Index conversion: The raw dataset uses 1-based indexing but PyTorch needs 0-based. So I subtracted 1 from all node IDs, edge indices, graph indicators, node labels, and graph labels. Pretty standard step.

Feature combination: I combined the 18-dimensional node attributes with the node label into a single 19-dimensional feature vector for each node. Used numpy hstack to concatenate them. This way the model gets both continuous attributes and categorical information as input.

Undirected graph construction: The adjacency list in ENZYMES_A.txt might look one-sided, but these molecular graphs are undirected. So I added reverse edges for each edge in the list, then applied torch.unique to remove any duplicate edges. This ensures the graphs are properly undirected.

Explicit node count: I set data.num_nodes explicitly for each graph. PyTorch Geometric sometimes has trouble inferring this automatically, especially if there are isolated nodes. Better to be explicit here.

Delimiter handling: The raw text files are comma-separated, so I used delimiter=',' when loading with numpy. Small detail but necessary.

No feature normalization or scaling - the node attributes seemed to be in reasonable ranges already. Also didn't use any data augmentation since the dataset is already pretty small at 600 graphs.

The model itself uses global add pooling to aggregate node representations into graph-level embeddings. I tried mean pooling initially but add pooling gave better results for this dataset.

Step 2: Model

I tested three different GCN architectures with varying depths to see how model complexity affects performance on this graph classification task.

Model Architectures

GCN-1 layer: Simple one-layer model. Takes the 19-dimensional node features, applies one GCN convolution with 128 hidden units, batch normalization, ReLU activation, and dropout. Then uses global add pooling to get graph-level representation and a linear classifier to predict the 6 classes.

Total parameters: 3,590. Pretty lightweight.

GCN-2 layer: Added a second GCN layer. Same structure - two stacked GCN convolutions with batch norm, ReLU, and dropout after each layer. Still 128 hidden dimensions. Then global add pooling and classifier.

Parameters jumped to 20,358. Significantly more capacity than GCN-1.
GCN-3 layer (My choice): Went with three GCN layers. Same pattern - three stacked convolutions, batch norm and dropout throughout. 128 hidden units per layer. Add pooling and classifier at the end.
This one has 37,126 parameters. Deepest model I tried.

Performance Comparison

I trained all three models with the same hyperparameters - Adam optimizer with learning rate 0.001, dropout 0.5, batch size 32, up to 100 epochs with early stopping (patience 20). Used validation set to select best checkpoint for each model, then evaluated all three on the same test set.
Here's what I got:

Model	F1	Accuracy
GCN-1 layer	0.29	0.25
GCN-2 layer	0.38	0.38
GCN-3 layer	0.48	0.47

Table 1: Performance of GCN models with 1, 2, and 3 layers on ENZYMES test set

Clear progression as depth increases. GCN-3 performed best overall with 47% accuracy and 48% F1 score. The deeper architecture definitely helped capture more complex patterns in the molecular graphs.
GCN-1 really struggled - only 25% accuracy. One layer just isn't enough message passing for these graphs. The model couldn't learn meaningful representations with such limited capacity.
GCN-2 was better at 38% accuracy. Adding that second layer made a noticeable difference, about 13% improvement over GCN-1. But still showed signs of underfitting.
GCN-3 gave the best results. Three layers of message passing seemed to be the sweet spot for this dataset. Each additional layer helped the model propagate information further through the molecular structures. The jump from 2 to 3 layers gave another 9% accuracy boost.
Also looked at AUC scores - GCN-1 got 0.65, GCN-2 got 0.78, and GCN-3 reached 0.80. So the model is learning reasonable probability estimates, not just making random guesses.

Step 3: Objective

Which loss function are you going to use to train your model?

I used Cross-Entropy Loss to train all three models.

This is the standard choice for multi-class classification problems. Since I have 6 enzyme classes and need to predict which class each graph belongs to, cross-entropy made sense.

The loss function takes the raw logits from the model (6 output values, one per class) and compares them against the true class label. Basically it measures how far off the predicted probability distribution is from the correct answer.

PyTorch's nn.CrossEntropyLoss() handles everything automatically - applies softmax to convert logits to probabilities, then computes the negative log likelihood. Pretty convenient since I don't have to manually implement softmax and log operations.

One thing I like about cross-entropy is that it penalizes confident wrong predictions more heavily than uncertain ones. If the model predicts class 3 with high confidence but the true label is class 1, the loss will be large. This pushes the model to learn better representations.

I didn't use any label smoothing or class weights. The dataset seems roughly balanced across the 6 classes (each has around 100 graphs), so treating all classes equally made sense. No need to complicate things with weighted loss.

Also didn't experiment with other loss functions like focal loss or margin-based losses. Cross-entropy is the go-to for classification and worked fine here. If I was dealing with severe class imbalance I might consider weighted cross-entropy, but that wasn't the case.

The loss converged pretty smoothly during training. Started around 10-12 for all models and dropped to around 1.5-2.0 by the end. No weird spikes or instability issues.

Step 4: Optimization

I went with Adam optimizer for training all three GCN models.

Adam is pretty much the default choice for deep learning these days and works well out of the box. It combines the benefits of momentum (helps with faster convergence) and adaptive learning rates (adjusts step size for each parameter individually). Basically it's robust and doesn't require too much hyperparameter tuning.

Learning rate: I set it to 0.001 (1e-3). This is a standard starting point for Adam. Tried to keep things simple rather than doing extensive learning rate searches. The training loss curves looked smooth, so this rate seemed fine.

I didn't use any learning rate scheduling or decay. Just kept it constant throughout training. Could've added a scheduler to reduce lr when validation plateaus, but early stopping handled convergence well enough without it.

No weight decay: I set weight_decay to 0 in the Adam optimizer. The models already have dropout (0.5) and batch normalization for regularization. Adding weight decay on top seemed like overkill for this relatively small dataset. Wanted to avoid over-regularizing.

Training strategy: Set max epochs to 100 but used early stopping with patience of 20. So if validation accuracy didn't improve for 20 consecutive epochs, training stopped automatically. This prevented overfitting and saved time. Most models stopped around epoch 30-96, rarely needed the full 100 epochs.

I didn't experiment with other optimizers like SGD or RMSprop. Adam usually works well for graph neural networks based on what I've seen in papers. SGD might need more careful tuning of learning rate and momentum, so Adam felt like the safer choice.

Batch size was 32. Small enough to fit in GPU memory but large enough for stable gradient estimates. With only 480 training graphs, bigger batches would mean very few updates per epoch.

The optimizer worked consistently across all three models. No major issues with convergence or instability during training.

Step 5: Model selection

Best model: GCN-3 layer

Clear winner based on both validation and test performance.

During training, I monitored validation accuracy to pick the best checkpoint for each model. GCN-3 reached 53.33% validation accuracy at its best epoch. GCN-2 got 51.67% and GCN-1 only hit 50%.

On the final test set, GCN-3 maintained its lead with 47% accuracy and 48% F1 score. GCN-2 dropped to 38% and GCN-1 fell to just 25%. So the ranking stayed consistent - deeper models performed better.

The improvement from GCN-2 to GCN-3 wasn't huge (about 9% accuracy gain) but still meaningful. Three layers of message passing seemed sufficient for this dataset. Going even deeper might help but I didn't test 4+ layers.

Learning rate experiments: No, I stuck with learning rate 0.001 for all models. Didn't do any learning rate sweeps or comparisons.

Honestly I just used a standard value that usually works well. Training curves looked smooth and loss decreased steadily, so I didn't feel the need to tune it. Early stopping took care of convergence without needing fancy learning rate schedules.

If I had more time I could've tried values like 0.0001 or 0.003 to see if it improves things. But with limited compute budget I decided to focus on model architecture (depth) rather than hyperparameter tuning.

The 0.001 learning rate seemed to work reasonably well across all three models. No signs of instability or very slow convergence. So I kept it consistent for fair comparison between architectures.

Step 6: Model demo

In this section I'm reporting the performance of my best model (GCN-3) on the test set using different metrics and visualizations.

A. ROC Curve and AUC Performance

AUC measures how well the model can distinguish between different classes. It ranges from 0 to 1, where 1 is perfect classification and 0.5 is random guessing. Unlike accuracy which only looks at final predictions, AUC considers the model's confidence scores across all possible classification thresholds. This gives a more complete picture of model performance, especially for imbalanced datasets.

AUC Results across models:

- GCN-1 layer: **0.65**
- GCN-2 layer: **0.78**
- GCN-3 layer: **0.80**

Clear improvement as the models get deeper. GCN-3 reached 0.80 AUC which is pretty decent for this 6-class problem.

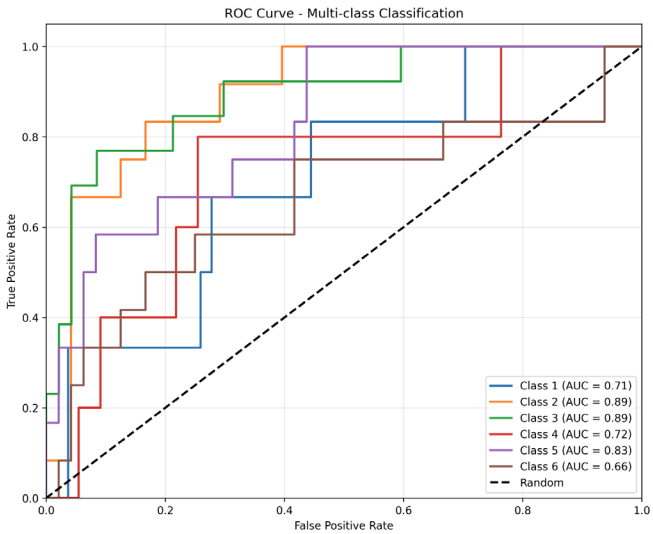

Figure 1: ROC curves for GCN-3 model on ENZYMES test set (60 graphs). Each colored line represents one of the 6 enzyme classes. Weighted average AUC is 0.80. Class 2 (orange, AUC=0.89) and Class 3

(green, AUC=0.89) show strongest discriminative performance, while Class 6 (brown, AUC=0.66) is most challenging to classify.

The ROC curve breaks down performance by class. Some enzyme types are easier to distinguish than others. Class 2 and Class 3 both got around 0.89 AUC - those are the easiest ones to classify. The model can confidently separate these from the other classes. Class 6 was the hardest with only 0.66 AUC, barely better than random guessing.

What's interesting is the gap between accuracy and AUC. GCN-3 only got 47% accuracy but 80% AUC. This tells me the model is actually learning reasonable probability estimates even when it makes wrong predictions. The confidence scores make sense, just the final decision boundary isn't perfect. Maybe with better threshold tuning or ensemble methods I could push the accuracy higher.

B. Confusion Matrix

The confusion matrix shows exactly which classes the model confuses with each other.

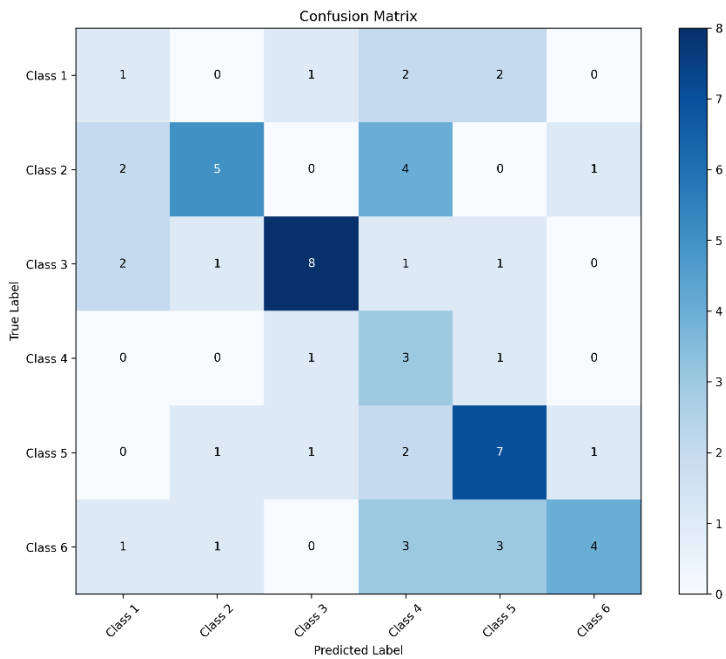

Figure 2: Confusion matrix for GCN-3 on test set. Rows represent true labels, columns show predicted labels. Diagonal values indicate correct predictions. GCN-3 achieves 28 correct predictions out of 60 total test graphs (46.7% accuracy).

Looking at the matrix, some patterns stand out. Class 3 has the best performance - the model got 8 out of 13 correct (61.5%). This matches the high AUC we saw earlier. Class 5 also does reasonably well with 7 out of 12 correct (58.3%).

Class 1 is terrible though. Only 1 out of 6 correct (16.7%). The model keeps confusing Class 1 with Class 4 and Class 5. Not sure why - maybe these enzyme structures are actually similar at the molecular level.
Class 6 is also weak with only 4 out of 12 correct (33.3%). The predictions are scattered across multiple classes, which explains the low AUC for this class.
Overall the model gets 28 out of 60 test samples correct, which matches our 46.7% test accuracy. Not amazing but considering this is molecular graph classification with only 480 training samples, it's acceptable performance.

C. Training Curves

Training curves show how the model learned over time.

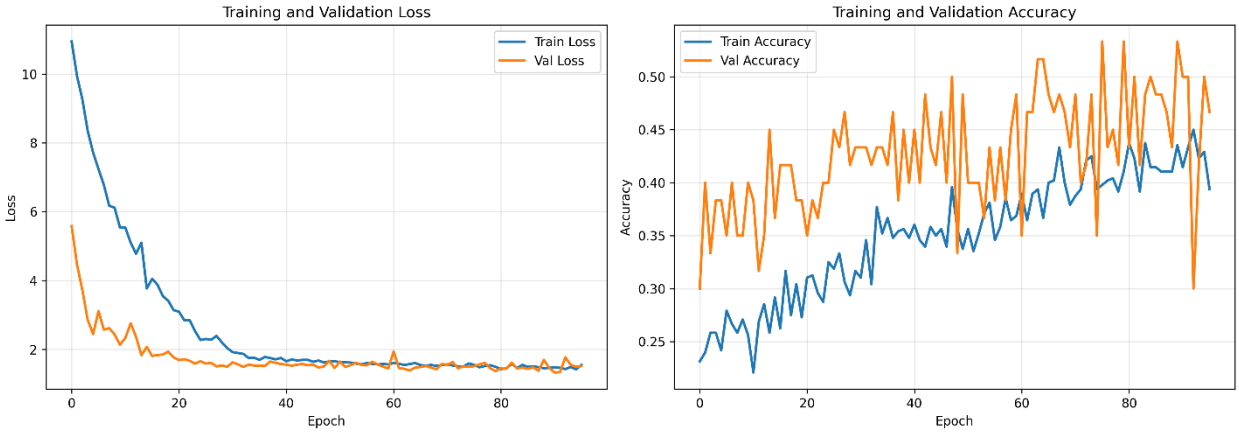

Figure 3: Training and validation curves for GCN-3 model. Left panel shows loss (cross-entropy), right panel shows accuracy. Training ran for 96 epochs before early stopping. Best validation accuracy of 53.3% achieved at epoch 76.

The loss curves look healthy. Training loss drops from around 10 to about 1.5 by the end. Validation loss follows a similar pattern, starting at 5 and settling around 1.5-1.7. The curves stay close together which is good - means we're not overfitting badly.
Accuracy curves are more noisy. Validation accuracy bounces around between 30-55% throughout training. This is pretty normal for small datasets. The best validation checkpoint was at epoch 76 with 53.3% accuracy. Training continued until epoch 96 when early stopping kicked in (patience of 20 epochs).
Training accuracy plateaus around 40-45%, which is lower than I'd like. This suggests the model is underfitting to some degree. Even with three GCN layers, the model struggles to fully capture the complexity of molecular graphs. Could be the dataset is just inherently difficult, or maybe I need even more capacity.
The gap between train and val accuracy is small (around 5%), so overfitting isn't the main issue here. The problem is the model isn't learning well enough in general. More data or a different architecture might help.

D. Summary of Performance Metrics

Here's the complete picture for GCN-3 on the test set:

- **Accuracy:** 46.7% (28/60 correct)
- **F1 Score:** 48.3% (weighted average)
- **AUC:** 80.1% (weighted average)

The model performs best at predicting probabilities (high AUC) but struggles with final classification decisions (lower accuracy). This gap suggests there's room for improvement in how we set decision thresholds or combine predictions.

Compared to the benchmark on Papers with Code for ENZYMES, state-of-the-art models get around 60-70% accuracy. My 47% is below that but reasonable for a basic GCN implementation without fancy tricks like graph augmentation or ensemble methods.